

UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE CIENCIAS
ESCUELA PROFESIONAL DE CIENCIAS DE LA
COMPUTACIÓN



COMPUTACIÓN GRÁFICA - CC431A

Informe N°3 - PC5:

**Integración de modelos de detección y estimación de
profundidad para mejorar sistemas de videovigilancia
inteligente**

Integrantes:

- Pumayauli Evangelista, Bryan José - 20212209E

- López Arteaga, César Omar - 20236743A

Profesor: Mg. Peter Jonathan Montalvo Garcia

Contenido

1. Aspectos Generales	3
1.1. Motivacion del Proyecto	3
1.1.1. Problema encontrado	3
1.1.2. Justificación	3
1.1.3. Propuesta de solución	3
1.2. Objetivo del Proyecto	4
1.2.1. Objetivo General	4
1.2.2. Objetivos Específicos	5
2. Fundamento Teórico	6
2.1. Yolo: modelo de aprendizaje profundo para detección de objetos en tiempo real	6
2.2. Depth Anything: modelo fundacional de aprendizaje profundo para estimación monocular de profundidad en visión por computadora	7
2.3. Profundidad relativa y decision de cercania	8
3. Implementación de Hardware	9
3.1. Implementación Hardware para prototipado	9
3.1.1. Materiales	9
3.1.2. Armado del prototipo	12
4. Estructura del proyecto implementado	16
4.1. Organizacion de carpetas	16
4.2. Descripcion funcional de los programas	17
4.3. Arquitectura general del sistema	17
5. Implementacion de los modulos principales	18
5.1. Modulo DepthAnythingONNX_RPi.py	18
5.2. Modulo SensorMovimientoFotoCorreoProfundidad.py	18
5.3. Modulo SensorMovimientoYoloProfundidadCorreo.py	19
5.4. Configuracion de correo	21

6. Algoritmo del sistema final	22
7. Pruebas realizadas y validacion	22
7.1. Pruebas unitarias	22
7.2. Resultados esperados del sistema final	23
7.3. Consideraciones sobre falsos positivos del PIR	23
8. Discusion tecnica	24
8.1. Justificacion del uso de dos resoluciones	24
8.2. Formato NCNN para YOLO	24
8.3. Formato ONNX para Depth Anything	24
8.4. Limitacion de escala metrica	25
9. Conclusiones	25
10.Recomendaciones	25

1. Aspectos Generales

1.1. Motivacion del Proyecto

1.1.1. Problema encontrado

En la actualidad, muchos sistemas de vigilancia están orientados principalmente al monitoreo continuo de ambientes, por lo que requieren la intervención constante de personal encargado de supervisar las cámaras, identificar posibles situaciones de riesgo y tomar acciones correctivas. Esta dependencia del factor humano puede generar retrasos en la detección de eventos peligrosos, aumentar los costos operativos y limitar la capacidad de respuesta inmediata ante incidentes.

1.1.2. Justificación

La mejora de los sistemas de vigilancia resulta necesaria para automatizar la detección de situaciones de riesgo, optimizar los tiempos de respuesta y reducir los costos asociados a la supervisión permanente. Mediante el uso de modelos de inteligencia artificial aplicados a visión por computadora, es posible implementar un sistema capaz de identificar personas, estimar su proximidad respecto a una zona de interés y generar alertas automáticas cuando se detecte una condición de peligro.

1.1.3. Propuesta de solución

La presente propuesta surge de la necesidad de implementar un sistema de vigilancia inteligente basado en modelos modernos de inteligencia artificial aplicados a visión por computadora. El sistema propuesto busca complementar el esquema clásico de monitoreo mediante la incorporación de sensores de movimiento y modelos de IA especializados en la detección de personas y la estimación de profundidad.

Para ello, se plantea utilizar un modelo de detección de objetos, como YOLO, encargado de identificar la presencia de personas en la escena, junto con un modelo de estimación monocular de profundidad, como Depth Anything, que permitirá estimar la distancia relativa de los objetos detectados respecto a la cámara.

El sistema permitirá detectar la presencia de personas en una zona específica y estimar su proximidad respecto a un área de interés. La alerta de seguridad se activará únicamente cuando la persona detectada se encuentre dentro de un límite previamente definido. En dicho caso, el sistema enviará una notificación por correo electrónico, adjuntando una imagen del evento detectado, así como la fecha y hora de ocurrencia.

Al tratarse de un sistema automatizado de monitoreo, la cámara y el equipo de procesamiento deberán instalarse en una posición fija dentro del ambiente a vigilar. De esta manera, se podrá realizar un seguimiento constante de la escena y generar alertas oportunas ante posibles situaciones de riesgo.

Para establecer un punto de comparación frente al modelo YOLO, se implementará también el modelo Faster R-CNN sobre la Raspberry Pi 4B. Esta comparación permitirá evaluar el desempeño de ambos enfoques de detección de objetos bajo las mismas condiciones de hardware, considerando métricas como el tiempo de inferencia, la cantidad de cuadros procesados por segundo, el consumo de memoria y la estabilidad de la detección. Faster R-CNN se considera un modelo adecuado para la comparación porque pertenece a la familia de detectores de dos etapas, donde primero se generan regiones candidatas mediante una *Region Proposal Network* y luego se realiza la clasificación y el ajuste de las cajas delimitadoras. La comparación permitirá analizar el compromiso entre precisión y velocidad, especialmente en un entorno de recursos limitados como la Raspberry Pi 4B.

1.2. Objetivo del Proyecto

1.2.1. Objetivo General

Implementar un sistema inteligente de vigilancia basado en modelos de inteligencia artificial para la detección de personas y la estimación de profundidad en una zona específica. Para el desarrollo y las pruebas del proyecto se empleará el lenguaje de programación Python, junto con bibliotecas de visión por computadora como OpenCV, integrando modelos de detección de objetos y estimación monocular de profundidad.

1.2.2. Objetivos Específicos

- Implementar un modelo YOLO para la detección de personas en imágenes o videos capturados en tiempo real.
- Implementar el modelo Depth Anything para estimar la profundidad relativa de los objetos presentes en la escena.
- Integrar la información obtenida por YOLO y Depth Anything para determinar la proximidad de las personas respecto a una zona de interés.
- Definir un límite de seguridad que permita establecer cuándo una persona detectada representa una posible situación de riesgo.
- Diseñar un sistema de alerta que notifique automáticamente cuando una persona se encuentre dentro del límite de seguridad previamente establecido.
- Incorporar el envío de alertas mediante correo electrónico, incluyendo una imagen del evento detectado y la fecha y hora de ocurrencia.
- Evaluar el funcionamiento del sistema mediante pruebas experimentales utilizando una cámara fija.
- Realizar el despliegue y las pruebas del sistema en una Raspberry Pi 4B.

2. Fundamento Teórico

2.1. Yolo: modelo de aprendizaje profundo para detección de objetos en tiempo real

YOLO es una arquitectura de red neuronal convolucional (CNN) de vanguardia diseñada específicamente para la detección de objetos en tiempo real. A diferencia de los detectores tradicionales de dos etapas (como Faster R-CNN, que primero proponen regiones y luego clasifican), YOLO pertenece a la familia de detectores de una sola etapa (single-stage detectors). Esto significa que enmarca la detección de objetos como un problema de regresión único, procesando la imagen entera en una sola evaluación (o "mirada") para predecir directamente las cajas delimitadoras (bounding boxes) y las probabilidades de clase.

El funcionamiento interno de YOLO se basa en dividir la imagen de entrada en una cuadrícula de $S \times S$. Si el centro de un objeto cae dentro de una celda específica de la cuadrícula, esa celda se vuelve responsable de detectar dicho objeto. Cada celda predice múltiples cajas delimitadoras junto con un puntaje de confianza que refleja qué tan seguro está el modelo de que la caja contiene un objeto, así como la precisión de las dimensiones de dicha caja.

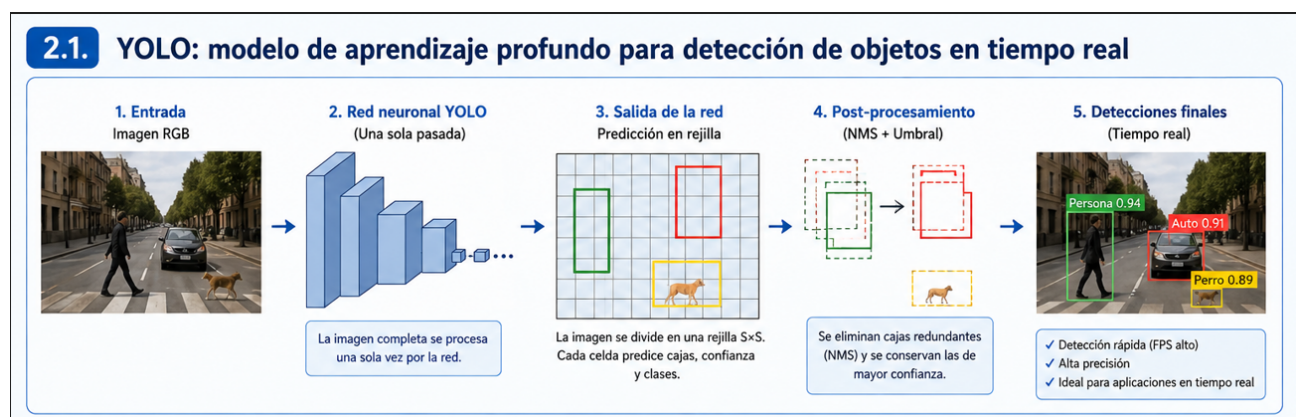


Figura 1: Yolo

2.2. Depth Anything: modelo fundacional de aprendizaje profundo para estimación monocular de profundidad en visión por computadora

La estimación monocular de profundidad (MDE) es una tarea fundamental en la visión por computadora que consiste en predecir la geometría 3D (la distancia de los objetos) a partir de una única imagen 2D. Tradicionalmente, esta tarea es considerada un problema "mal planteado" (ill-posed), ya que una misma imagen bidimensional puede provenir de infinitas escenas tridimensionales.

Depth Anything es un modelo fundacional (foundation model) altamente robusto diseñado para resolver el problema de la estimación de profundidad monocular. Este modelo se destaca por su capacidad de generalización en modo zero-shot, lo que significa que puede estimar la profundidad con alta precisión en escenas e imágenes que nunca vio durante su entrenamiento, sin necesidad de ajustes finos (fine-tuning).

El éxito de Depth Anything radica en su proceso de entrenamiento a gran escala y su arquitectura basada típicamente en Transformers (como DINOv2). Para superar las limitaciones de los conjuntos de datos pequeños o sesgados, este modelo se entrena combinando una cantidad masiva de imágenes etiquetadas (con datos de profundidad reales capturados por sensores como LiDAR) y un volumen aún mayor de imágenes sin etiquetar mediante técnicas de aprendizaje semi-supervisado o auto-supervisado. Esto le permite capturar una comprensión semántica y estructural excepcionalmente rica de cualquier escena.

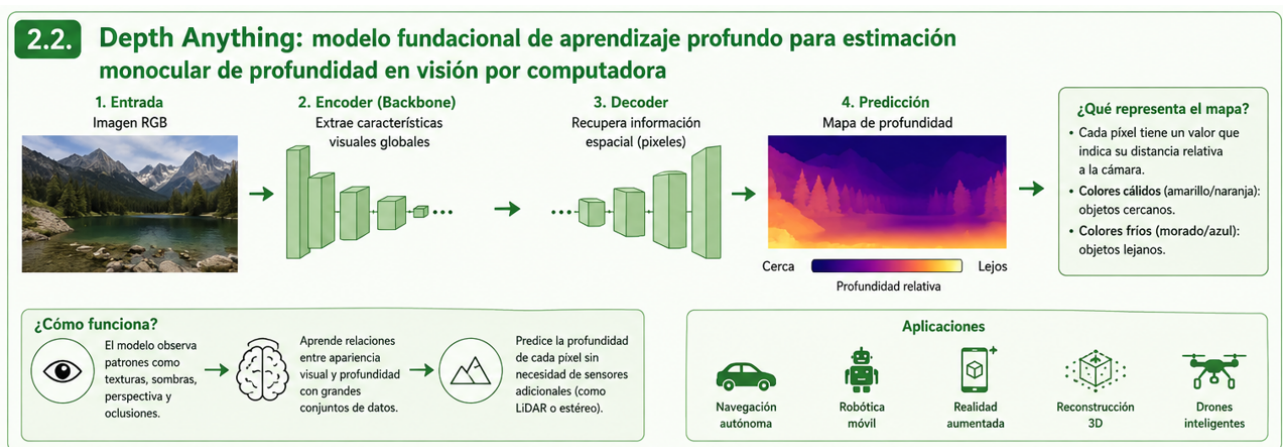


Figura 2: Depth Anything

2.3. Profundidad relativa y decision de cercania

Sea $I \in \mathbb{R}^{H \times W \times 3}$ el frame capturado por la camara. Depth Anything genera un mapa

$$D \in \mathbb{R}^{H \times W}, \quad (1)$$

donde $D(x, y)$ representa la profundidad relativa del pixel (x, y) . En el codigo, este mapa se normaliza a $[0, 1]$ mediante

$$D_n(x, y) = \frac{D(x, y) - D_{\min}}{D_{\max} - D_{\min} + \varepsilon}. \quad (2)$$

Si YOLO detecta una persona mediante la caja

$$B = (x_1, y_1, x_2, y_2), \quad (3)$$

se calcula una region interna Ω_B para evitar bordes y fondo. Luego se usa la mediana de profundidad relativa:

$$z_B = \text{mediana} \{D_n(x, y) : (x, y) \in \Omega_B\}. \quad (4)$$

La alerta de proximidad se activa si

$$z_B \geq \tau, \quad (5)$$

con $\tau = 0,62$ en la configuracion inicial del codigo. Este umbral debe calibrarse experimentalmente segun posicion de camara, iluminacion y zona vigilada.

3. Implementación de Hardware

3.1. Implementación Hardware para prototipado

En esta sección se describe la construcción física del prototipo utilizado para el sistema de videovigilancia inteligente. El prototipo integra una Raspberry Pi 4B como unidad principal de procesamiento, una cámara Raspberry Pi NoIR para la captura de imágenes, un sensor de movimiento HC-SR501 como apoyo para la activación del sistema y elementos de protección y refrigeración para asegurar un funcionamiento estable durante las pruebas.

3.1.1. Materiales

Para el desarrollo del proyecto se emplean componentes electrónicos orientados a captura de video, procesamiento local e interacción con el entorno. Cada elemento cumple una función específica dentro del prototipo, ya sea para adquirir imágenes, ejecutar los modelos de inteligencia artificial, detectar movimiento o mejorar la estabilidad térmica del sistema.

- **Raspberry Pi 4B de 8 GB de RAM.** Es la placa principal del prototipo y se encarga de ejecutar el sistema operativo, capturar los datos de la cámara, procesar las imágenes y ejecutar los programas desarrollados en Python. Sobre esta placa se implementarán los modelos de visión por computadora, como YOLO para la detección de personas y Depth Anything para la estimación de profundidad.



Figura 3: Raspberry Pi 4B utilizada como unidad principal de procesamiento del prototipo.

- **Case Argon Neo Pi 4B.** Se utiliza como carcasa de protección para la Rasp-



Figura 6: Sensor de movimiento HC-SR501 usado como apoyo para detectar presencia o movimiento en la zona vigilada.

- **Ventilador Argon40 Mini FAN.** Este ventilador se incorpora para mejorar la refrigeración de la Raspberry Pi. Su función es reducir el calentamiento durante la ejecución de procesos de visión por computadora, ya que los modelos de detección y estimación de profundidad pueden incrementar la carga computacional del sistema.



Figura 7: Ventilador Argon40 Mini FAN utilizado para mejorar la disipación térmica del prototipo.

- **MicroSD de 32 GB clase 10.** Se utiliza como medio de almacenamiento principal de la Raspberry Pi. En ella se instala el sistema operativo, las bibliotecas necesarias, los scripts de Python y los archivos asociados al funcionamiento del prototipo.

3.1.2. Armado del prototipo

El armado del prototipo consiste en integrar la Raspberry Pi dentro del case, fijar la cámara en una posición estable, conectar el sensor de movimiento y organizar los cables de alimentación y comunicación. La finalidad de este montaje es obtener una estructura compacta que pueda ubicarse frente a una zona específica para realizar pruebas de videovigilancia inteligente.

La conexión de la cámara se realizará mediante el bus dedicado CSI de la Raspberry Pi 4B, el cual permite la comunicación directa entre la placa y el módulo de cámara compatible. Esta interfaz resulta adecuada para aplicaciones de visión por computador, ya que permite capturar imágenes o video con menor latencia en comparación con una cámara conectada por USB.

Para la conexión del ventilador se utilizarán los pines de alimentación de la Raspberry Pi 4B, considerando el nivel de voltaje requerido por el dispositivo. En caso de emplear un ventilador de 5 V, este deberá conectarse a un pin de 5 V y a un pin GND. Si se requiere controlar el encendido y apagado del ventilador desde el programa, no debe conectarse directamente a un pin GPIO, sino mediante una etapa de control, como un transistor o un módulo MOSFET, debido a que los pines GPIO no están diseñados para alimentar cargas de corriente elevada.

La conexión del sensor de movimiento se realizará mediante los pines GPIO de la placa. La señal de salida del sensor PIR se conectará al GPIO 23 de la Raspberry Pi 4B, que corresponde al pin físico 16. La alimentación del sensor se tomará desde la salida de 3,3 V de la placa, mientras que el terminal GND del sensor se conectará a uno de los pines de tierra de la Raspberry Pi. De esta manera, el sistema podrá detectar movimiento mediante la lectura digital del GPIO 23 y activar los procesos correspondientes dentro del sistema de videovigilancia.

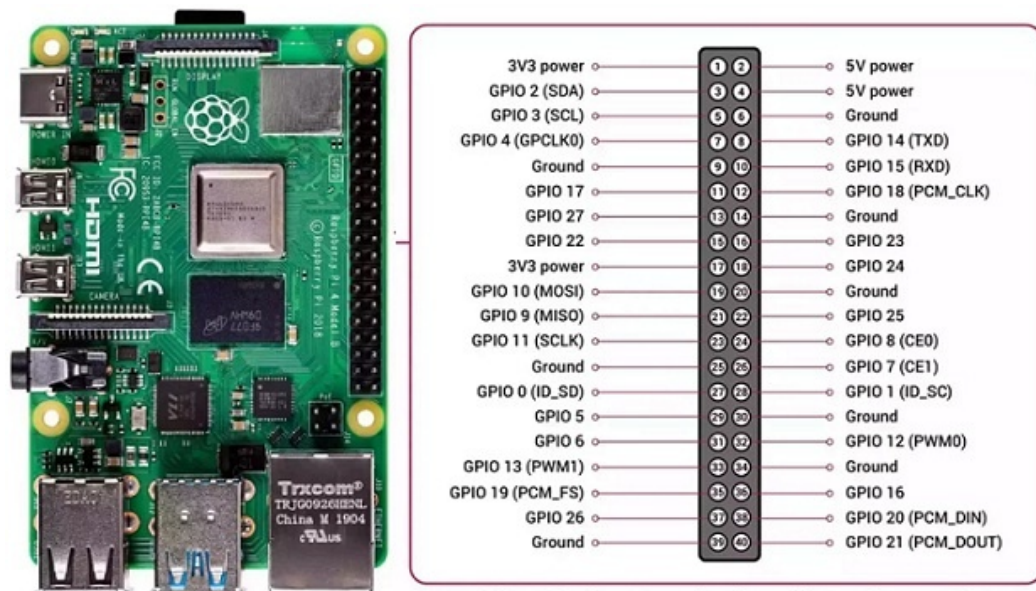


Figura 8: Vista frontal del diagrama de pines de la Raspberry Pi 4B.

En la vista frontal del prototipo se observa la disposición principal del sistema. La cámara se encuentra ubicada en la parte superior, orientada hacia la zona de monitoreo, mientras que el sensor de movimiento se coloca al frente para detectar presencia o desplazamiento. Esta configuración permite que la cámara capture la escena y que el sensor actúe como apoyo para la activación del sistema.



Figura 9: Vista frontal del prototipo, donde se aprecia la ubicación de la cámara y del sensor de movimiento sobre la estructura de la Raspberry Pi.

En la vista lateral se aprecia con mayor claridad la conexión física entre la cámara, el sensor y la Raspberry Pi. Esta vista permite verificar la distribución de los cables y la

forma en que los componentes quedan sujetos al case. La organización del cableado es importante para evitar desconexiones durante las pruebas y para mantener estable la orientación de la cámara.

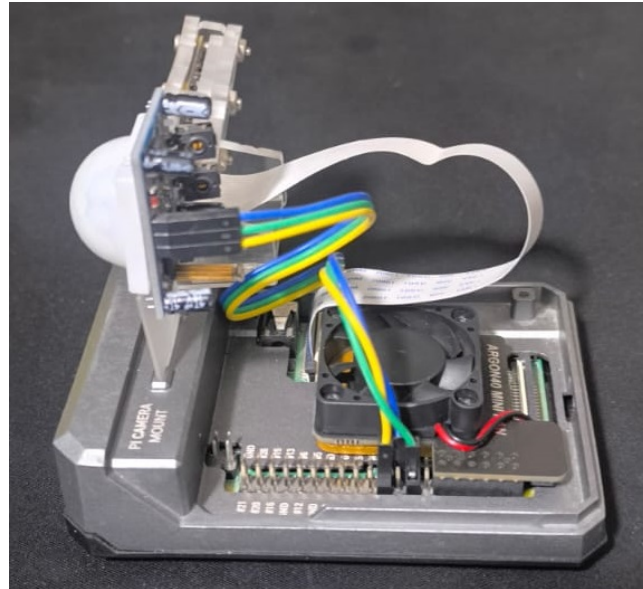


Figura 10: Vista lateral del prototipo, mostrando la conexión de la cámara, el sensor de movimiento y el cableado hacia la Raspberry Pi.

En la vista en perspectiva se observa la estructura completa del prototipo. Esta imagen permite identificar la integración general de los elementos: la Raspberry Pi como unidad central, la cámara como sistema de captura y el sensor como módulo complementario de detección de movimiento. El montaje busca mantener todos los componentes en una posición fija para obtener mediciones y capturas consistentes.

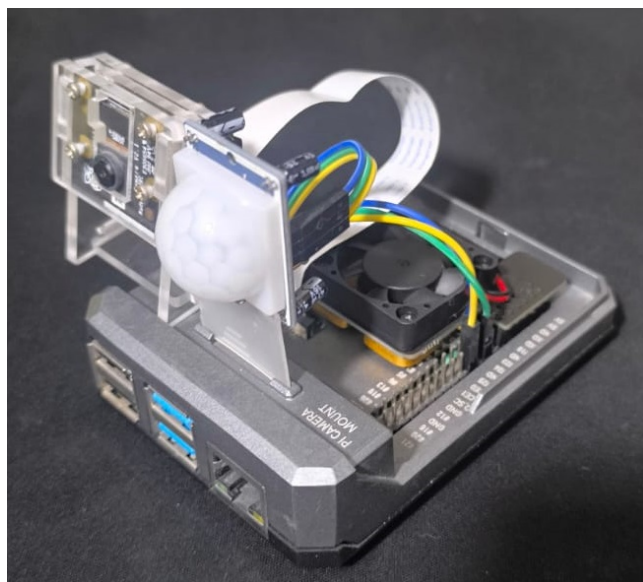


Figura 11: Vista en perspectiva del prototipo ensamblado, donde se muestra la integración general de la Raspberry Pi, la cámara y el sensor.

Finalmente, la implementación del prototipo en un ambiente de prueba permite evaluar su funcionamiento en una escena real. En esta etapa se coloca el sistema frente a una zona de observación para comprobar la captura de imágenes, la estabilidad del montaje y la posibilidad de procesar la información obtenida mediante los modelos de inteligencia artificial planteados para el proyecto.



Figura 12: Implementación del prototipo en un ambiente de prueba para validar la captura de imágenes y el funcionamiento inicial del sistema de monitoreo.

4. Estructura del proyecto implementado

4.1. Organizacion de carpetas

Los codigos entregados se organizaron en modulos progresivos. Primero se validan camara y sensor de forma separada; luego se incorporan correo, YOLO, Depth Anything y finalmente la integracion completa.

```
1 YOLO_PROFUNDIDAD_RASPBERRYPI4B/  
2 |-- manejo_camara/  
3 |   |-- foto_con_camara.py  
4 |   '-- video_con_camara.py  
5 |-- manejo_sensor/  
6 |   |-- SensorMovimiento.py  
7 |   '-- CamaraMovimientoFecha.py  
8 |-- yolo_sensor_camara/  
9 |   |-- SensorMovimientoCorreo.py  
10 |   '-- SensorMovimientoFotoCorreo.py  
11 |-- yolo_deteccion/  
12 |   |-- deteccion_camara_imagen_chico.py  
13 |   |-- deteccion_camara_imagen_grande.py  
14 |   |-- yolo11n.pt  
15 |   '-- yolo11n_ncnn_model/  
16 '-- yolo_profundidad/  
17     |-- DepthAnythingONNX_RPi.py  
18     |-- SensorMovimientoFotoCorreoProfundidad.py  
19     |-- SensorMovimientoYoloProfundidadCorreo.py  
20     '-- models/  
21         |-- depth_anything_v2_vits.onnx  
22         '-- yolo11n_ncnn_model/
```

Listing 1: Estructura logica del proyecto en la Raspberry Pi.

4.2. Descripción funcional de los programas

Cuadro 1: Programas principales y función dentro del proyecto.

Programa	Función
foto_con_camara.py	Prueba básica de captura de imagen con la cámara CSI conectada a la Raspberry Pi 4B.
video_con_camara.py	Prueba de visualización de video en tiempo real mediante Picamera2.
SensorMovimiento.py	Prueba de lectura digital del sensor PIR conectado al GPIO 23.
CamaraMovimientoFecha.py	Integra el sensor PIR con la cámara y registra la fecha y hora de la captura.
SensorMovimientoCorreo.py	Envía un correo electrónico cuando el sensor PIR detecta movimiento.
[]SensorMovimientoFoto Correo.py	Captura una fotografía y la envía por correo como evidencia del evento detectado.
[]deteccion_camara _imagen_chico.py	Ejecuta YOLO con una configuración liviana para la Raspberry Pi 4B.
[]deteccion_camara _imagen_grande.py	Ejecuta YOLO con mayor resolución de captura para pruebas visuales.
DepthAnythingONNX_RPi.py	Módulo auxiliar que carga Depth Anything V2 en formato ONNX y genera mapas de profundidad relativa.
[]SensorMovimientoFoto CorreoProfundidad.py	Integra sensor PIR, cámara, Depth Anything y correo. No utiliza YOLO.
[]SensorMovimientoYolo ProfundidadCorreo.py	Sistema final que integra sensor PIR, cámara, YOLO, Depth Anything, decisión por cercanía y envío de alerta por correo.

4.3. Arquitectura general del sistema

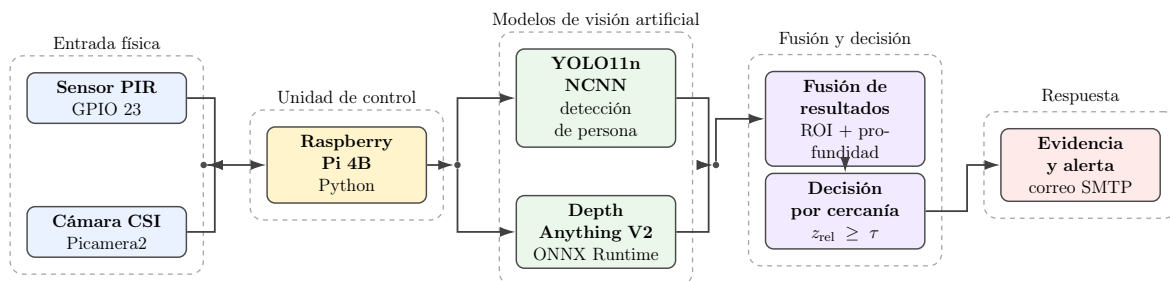


Figura 13: Arquitectura general del sistema de videovigilancia inteligente implementado en Raspberry Pi 4B.

5. Implementacion de los modulos principales

5.1. Modulo DepthAnythingONNX_RPi.py

El archivo DepthAnythingONNX_RPi.py encapsula la inferencia de profundidad. Su clase principal es DepthAnythingONNX. El constructor valida la existencia del modelo, crea una sesion ONNXRuntime en CPU, identifica el nombre de entrada/salida y lee automaticamente el tamaño real esperado por el modelo.

Este detalle es importante porque el modelo utilizado espera entrada fija 518×518 . Por ello, el codigo evita errores de dimension como Got: 256 Expected: 518. El preprocesamiento transforma la imagen de BGR a RGB, redimensiona al tamaño del modelo, normaliza usando media y desviacion estandar de ImageNet, y reorganiza las dimensiones de HWC a NCHW.

```
1 depth_model = DepthAnythingONNX(  
2     MODELO_DEPTH ,  
3     input_size=518 ,  
4     num_threads=2 ,  
5 )  
6  
7 depth_norm = depth_model.infer(frame_bgr)  
8 depth_color = colorizar_profundidad(depth_norm)
```

Listing 2: Carga de Depth Anything V2 con ONNXRuntime en CPU.

La funcion valor_roi_profundidad recibe el mapa de profundidad normalizado y una caja de YOLO. Para reducir ruido, calcula la mediana dentro de una region interna de la caja, descartando un margen del 15 % en los bordes.

5.2. Modulo SensorMovimientoFotoCorreoProfundidad.py

Este programa extiende el sistema de sensor y correo incorporando Depth Anything. Su objetivo es generar evidencia visual de movimiento con tres archivos:

1. Foto original anotada con fecha y hora.
2. Mapa de profundidad relativa coloreado.

3. Imagen comparativa foto original | profundidad.

Este modulo conserva la resolucion maxima usada en la captura fotografica:

$$3280 \times 2464 \text{ pixeles.} \quad (6)$$

El flujo se muestra en la Figura 14.

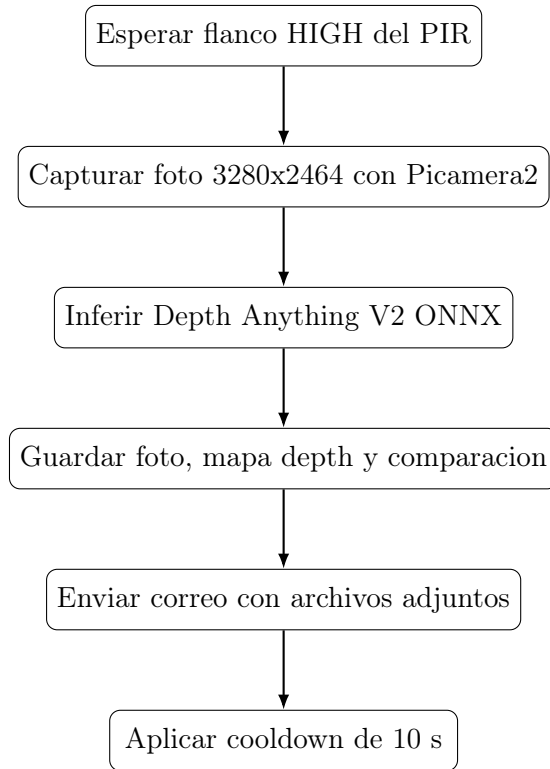


Figura 14: Flujo del programa SensorMovimientoFotoCorreoProfundidad.py.

5.3. Modulo SensorMovimientoYoloProfundidadCorreo.py

Este es el modulo final del proyecto. Integra sensor PIR, camara, YOLO, Depth Anything, fusion de resultados y correo. A diferencia del modulo anterior, aqui se usa resolucion moderada:

$$640 \times 480 \text{ pixeles,} \quad (7)$$

porque la inferencia combinada de YOLO y Depth Anything requiere menor carga computacional en la Raspberry Pi 4B.

El modelo YOLO se carga desde la carpeta `models/yolo11n_ncnn_model`. La detección se restringe a la clase `person`:

```

1 results = yolo_model.predict(
2     frame_bgr,
3     imgsz=320,
4     conf=0.35,
5     classes=[0],      # person en COCO
6     verbose=False,
7 )

```

Listing 3: Inferencia YOLO restringida a personas.

Luego se ejecuta Depth Anything y se calcula el valor z_B dentro de cada caja de persona. El programa dibuja la caja sobre la imagen, coloca la confianza de YOLO y el valor de profundidad relativa. Si la persona supera el umbral de cercanía, la caja se dibuja en rojo; en caso contrario se dibuja en verde.

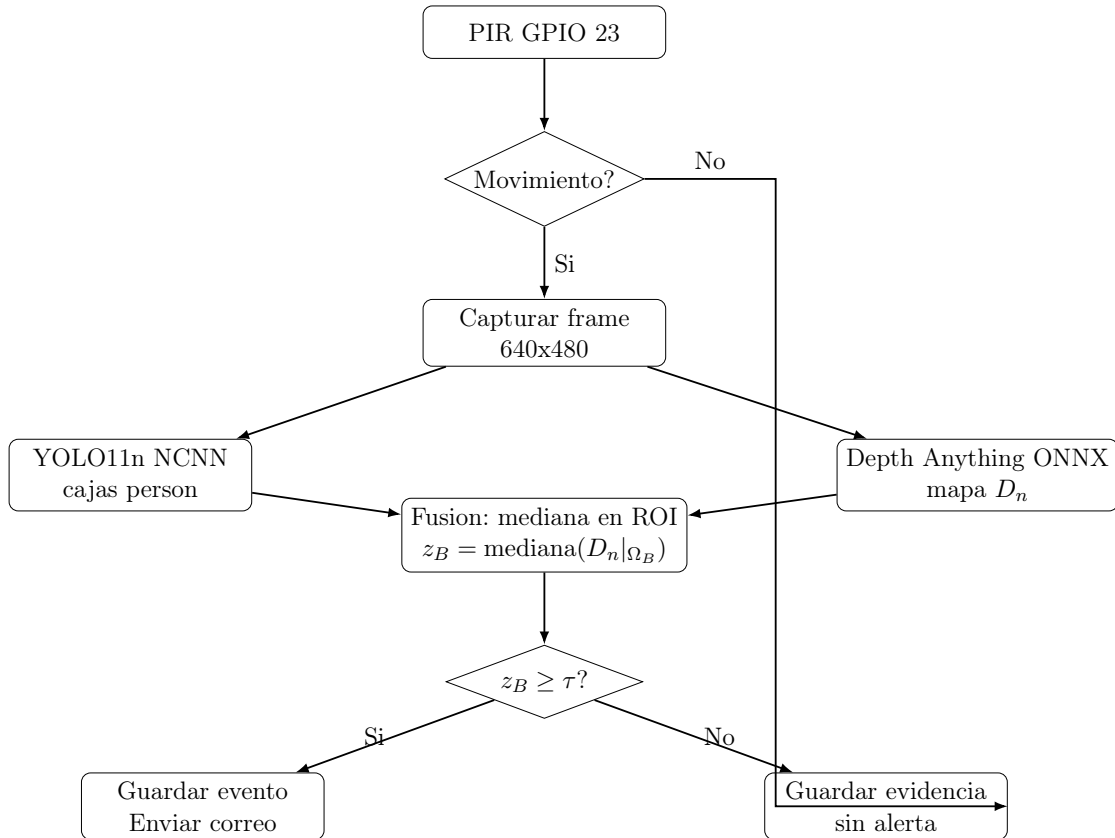


Figura 15: Flujo final de decision: PIR, YOLO, profundidad y correo.

5.4. Configuración de correo

Los programas leen las credenciales desde variables de entorno. Esto evita escribir usuario y contraseña directamente dentro del código fuente. Las variables utilizadas son:

```
1 export GMAIL_USER="correo_emisor@gmail.com"
2 export GMAIL_APP_PASSWORD="clave_de_aplicacion"
3 export GMAIL_TO="correo_destino@gmail.com"
```

Listing 4: Variables de entorno usadas para el correo.

El envío se realiza mediante SMTP seguro:

Servidor SMTP = *smtp.gmail.com*, Puerto = 465. (8)

6. Algoritmo del sistema final

Algorithm 1 Sistema final PIR + YOLO + Depth Anything + correo

```
1: Inicializar GPIO en modo BCM.
2: Configurar PIR en GPIO 23 con PUD_DOWN.
3: Inicializar camara con Picamera2 a  $640 \times 480$ .
4: Cargar YOLO11n en formato NCNN.
5: Cargar Depth Anything V2 en formato ONNX.
6: while el sistema esta activo do
7:   Leer estado actual del PIR.
8:   if existe flanco ascendente de LOW a HIGH then
9:     Capturar frame RGB desde la camara.
10:    Convertir frame RGB a BGR.
11:    Ejecutar YOLO restringido a clase persona.
12:    Ejecutar Depth Anything y obtener  $D_n$ .
13:    for cada persona detectada do
14:      Obtener caja  $B = (x_1, y_1, x_2, y_2)$ .
15:      Calcular region interna  $\Omega_B$ .
16:      Calcular  $z_B = \text{mediana}(D_n|_{\Omega_B})$ .
17:      if  $z_B \geq \tau$  then
18:        Marcar persona como cercana.
19:        Activar bandera de alerta.
20:      end if
21:    end for
22:    Guardar imagen comparativa YOLO + profundidad.
23:    if bandera de alerta activa then
24:      Enviar correo con imagen del evento.
25:      Aplicar tiempo de espera para evitar correos repetidos.
26:    end if
27:  end if
28: end while
```

7. Pruebas realizadas y validacion

7.1. Pruebas unitarias

La validacion del sistema se realizo por etapas. Este enfoque evita mezclar errores de hardware, librerias, camara, modelos y correo.

Cuadro 2: Plan de pruebas del sistema.

Prueba	Comando o modulo	Resultado esperado
Camara	foto_con_camara.py, video_con_camara.py	Captura y visualizacion e
PIR	SensorMovimiento.py	Cambio de estado al detectar
Correo	SensorMovimientoCorreo.py	Envio de notificacion al detecta
Foto + correo	SensorMovimientoFoto Correo.py	Envio de evidencia fotograf
YOLO	deteccion_camara _imagen_chico.py	Deteccion de personas co
Depth Anything	SensorMovimientoFoto CorreoProfundidad.py	Mapa de profundidad y compa
Fusion final	SensorMovimientoYolo ProfundidadCorreo.py	Alerta solo si hay persona

7.2. Resultados esperados del sistema final

Cuando el sistema final detecta un evento, genera una imagen comparativa con dos partes: a la izquierda se muestra el frame anotado por YOLO y a la derecha el mapa de profundidad relativa. La etiqueta de cada persona incluye la confianza de YOLO y el valor z estimado en la region de la caja.

$$\text{Etiqueta} = \text{personaconfianza}|z = \text{profundidad_relativa}. \quad (9)$$

Si $z \geq 0,62$, la persona se considera cercana y se envia correo. Si no se cumple el umbral, el evento puede guardarse como evidencia, pero no se considera alerta critica.

7.3. Consideraciones sobre falsos positivos del PIR

Durante las pruebas se observo que el PIR puede generar activaciones aun cuando no existe movimiento evidente. Esto se debe a que el PIR no detecta imagenes, sino variaciones de radiacion infrarroja. Por ello, puede reaccionar ante calor, corrientes de aire, cambios de luz, sensibilidad alta o modo retrigger.

El codigo implementado usa `pull_up_down=GPIO.PUD_DOWN` y detecta flancos ascendentes, lo cual reduce activaciones continuas. Sin embargo, para mayor robustez se recomienda ajustar fisicamente el sensor:

- Bajar el potenciómetro de sensibilidad.

- Reducir el tiempo de retardo del modulo PIR.
- Usar modo no retrigger si el jumper H/L lo permite.
- Mantener el sensor lejos de ventiladores, ventanas o fuentes de calor.
- Verificar que OUT vaya al GPIO 23 y que GND sea comun.

8. Discusion tecnica

8.1. Justificacion del uso de dos resoluciones

El proyecto usa dos resoluciones segun el objetivo del programa. En `SensorMovimientoFotoCorreoProfundidad.py` se conserva la captura de alta resolucion 3280×2464 , porque el proposito principal es generar evidencia visual de buena calidad. En `SensorMovimientoYoloProfundidadCorreo.py` se usa 640×480 , porque ejecutar YOLO y Depth Anything simultaneamente en Raspberry Pi 4B requiere reducir la carga de CPU y memoria.

8.2. Formato NCNN para YOLO

El modelo YOLO original `yolo11n.pt` se usa como base, pero la inferencia final en Raspberry Pi 4B se realiza con la carpeta `yolo11n_ncnn_model`. Esta carpeta contiene archivos `model.ncnn.param`, `model.ncnn.bin` y metadatos. El formato NCNN permite ejecutar la red de forma mas eficiente en CPU ARM.

8.3. Formato ONNX para Depth Anything

Depth Anything se ejecuta con `ONNXRuntime` usando `CPUExecutionProvider`. El codigo detecta automaticamente la dimension de entrada del modelo. En las pruebas, el modelo espera 518×518 , por lo que el preprocesamiento redimensiona cada imagen a ese tamaño antes de la inferencia.

8.4. Limitacion de escala metrica

La profundidad estimada es relativa. Para convertirla en metros seria necesario calibrar el sistema con distancias conocidas o usar informacion adicional. En el alcance del proyecto, el objetivo es determinar proximidad relativa, no medir distancia absoluta.

9. Conclusiones

1. Se implemento un prototipo funcional sobre Raspberry Pi 4B que integra sensor PIR, camara CSI, modelos de vision por computadora y envio de correo.
2. YOLO11n en formato NCNN permite detectar personas de forma mas adecuada para un entorno embebido que el modelo PyTorch original.
3. Depth Anything V2 en formato ONNX permite estimar un mapa de profundidad relativa sin requerir sensores de profundidad adicionales.
4. La fusion entre YOLO y Depth Anything permite evaluar la cercania dentro de la region donde se detecta una persona, evitando analizar toda la escena sin contexto.
5. El sistema final genera evidencia visual y envia alerta solamente cuando se cumple la condicion de persona cercana, lo cual mejora el criterio frente a un sistema basado solo en movimiento.
6. El PIR puede producir falsos positivos por causas fisicas; por ello, la decision final no debe depender solamente del sensor, sino de la combinacion con YOLO y profundidad.

10. Recomendaciones

- Calibrar el umbral τ colocando una persona a diferentes distancias y registrando el valor z mostrado por el sistema.
- Mantener la resolucion 640×480 para el modulo final si se prioriza estabilidad en Raspberry Pi 4B.

- Ajustar físicamente el PIR para reducir falsas activaciones.
- Separar las credenciales de correo del código fuente y mantenerlas como variables de entorno.
- Guardar ejemplos de eventos reales para anexarlos al informe como evidencia experimental.

Referencias

- [1] Yang, L., Kang, B., Huang, Z., Zhao, Z., Xu, X., Feng, J. y Zhao, H. (2024). *Depth Anything V2*. NeurIPS 2024.
- [2] Depth Anything. (2024–2026). *Repositorio oficial de Depth Anything V2*. GitHub.
- [3] Ultralytics. (2024–2026). *YOLO NCNN Export*. Documentación oficial de Ultralytics.
- [4] Raspberry Pi Ltd. (2025). *The Picamera2 Library Manual*.